Skill 核心概念

- 来源: https://class.imooc.com/lesson/5287#mid=105136
- 阶段: 阶段五: 【能力封装】可复用技能
- 周: 第10周 Skill：可复用技能封装
- 模块: Skill核心概念与扩展能力
- 课节: 图文 1-1 Skill 核心概念

1-1 Skill 核心概念

Skill 核心概念

> [!TIP] 阅读指南 上一节我们了解了 Skill 的生态和获取途径。现在，深入学习 Skill 的核心概念，掌握它的工作原理和设计哲学。 本章演示环境： 本章所有示例和操作步骤基于 Qoder 进行演示。

Skill 基础

Skill 的组成

一个完整的 Skill 由以下部分组成:

```
my-skill/                    ← Skill 目录
  ├─ SKILL.md                ← 核心文件(必需)
  │   ├─ YAML 元数据          ← 名称、描述
  │   └─ Markdown 正文        ← 详细指令
  │
  ├─ reference.md            ← 参考文档(可选)
  ├─ examples.md             ← 示例代码(可选)
  │
  ├─ scripts/                ← 工具脚本(可选)
  │   ├─ helper.py
  │   └─ formatter.sh
  │
  └─ templates/              ← 模板文件(可选)
      └─ template.txt
```

预览

- SKILL.md 是唯一必需的文件
- 其他文件都是可选的，用于补充说明
- 文件组织灵活，根据需要添加

Skill vs 其他概念

对比一下 Skill 和其他常见概念的区别:

SKILL.md 文件结构

现在详细学习 SKILL.md 的标准格式。

文件结构概览

```
---
# ============================================
#  YAML Frontmatter (元数据区)
# ============================================
name: skill-name
description: 简短描述,说明这个 Skill 做什么、何时使用
---

# ============================================
#  Markdown Body (正文区)
# ============================================

# Skill 名称

## 使用场景
说明什么情况下应该使用这个 Skill

## 核心指令
详细的操作步骤和规范

## 示例
具体的使用示例

## 参考资料
链接到其他相关文档
```

YAML Frontmatter 详解

YAML 元数据区包含 Skill 的基本信息：

```
---
name: python-best-practices
description: Python 开发最佳实践，包括代码规范、类型注解、文档字符串。当编写或审查 Python 代码时使用
---
```

标准 YAML 只有两个字段——name 和 description。以下逐一说明。

字段说明

```
name（必需）
──────────────────────────────────────────────────────────────
  • Skill 的唯一标识符
  • 只能使用：小写字母、数字、连字符
  • 长度限制：最多 64 个字符
  • 命名建议：使用描述性名称，如 code-review、git-commit

  ✓ 好的命名：
    python-dev
    api-design
    code-review

  × 不好的命名：
    Skill1（不具描述性）
    Python_Dev（不能用下划线）
    code review（不能有空格）
```

```
description（必需）
──────────────────────────────────────────────────────────────
```

- 这是 Skill 的触发入口——AI 靠它判断何时加载这个 Skill
- 长度限制：最多 1024 个字符
- 必须包含：
  1. 这个 Skill 做什么
  2. 何时应该使用（触发条件）
  3. 触发关键词（帮助 AI 匹配用户意图）

```
✓ 好的 description：
"专业的代码审查助手，检查安全漏洞、性能问题、代码规范。
 当用户要求审查、review、检查代码时使用"

× 不好的 description：
"帮助写代码"（太模糊，AI 不知道何时触发）

关键理解：description 是 AI 在决定「要不要加载这个 Skill」时
唯一能看到的信息。正文内容只有在 Skill 被触发后才加载。
因此，触发关键词必须写在 description 里，而不是写在正文中。

实际项目中通常直接嵌入触发标记：

"图书正文格式规范。用于编写/优化图书章节，确保：结构清晰、
 去AI味、符合出版标准。触发：(1)创建新章节 (2)修改现有文档
 (3)去除AI痕迹 (4)规范化格式"
```

早期 Skill 规范中还有一个实验性字段 allowed-tools，用于限制 Skill 激活后的工具权限。但当前主流标准已不再推荐此字段——AI 本身就能根据任务判断该用什么工具，不需要人为约束。一个标准的 Skill，YAML 区只需要 name 和 description。

Markdown 正文区详解

相对于YAML的严格，正文区比较随意，主要是用非结构化的文本描述Skill的内容，可以按需组织:

示例模板

```
# Python 开发最佳实践

## 使用场景
当编写、修改或审查 Python 代码时使用此 Skill

## 代码规范

### 格式化
- 使用 Black 自动格式化
- 行长度限制为 88 字符
- 使用 4 空格缩进

### 命名规范
- 类名: PascalCase (UserManager)
- 函数/变量: snake_case (get_user, user_id)
- 常量: UPPER_SNAKE_CASE (MAX_RETRY)

## 错误处理
- 明确捕获具体异常,避免裸 except
- 关键操作必须有 try-except
- 记录错误日志

## 参考资料
更多详情请参考 [advanced.md](advanced.md)
```

组织建议

```
好的 Skill 正文结构:
  ├─ 使用场景 (让 AI 知道何时用)
  ├─ 核心规则 (必须遵守的标准)
  ├─ 示例代码 (展示正确用法)
  ├─ 常见错误 (避免踩坑)
  └─ 参考链接 (深入学习)
```

如果想看一个真实项目中完整的 SKILL.md 文件，可以回顾上一节我们安装并使用的 Humanizer 技能——它就是一份高质量的 Skill 参考范本。

Skill 的触发机制

理解 Skill 如何被触发是很重要的。

自动触发与手动触发

Skill 的理想工作方式是自动触发。AI 根据用户说的话，在后台匹配 description 中的关键词，判断是否需要加载某个 Skill。

```
用户："帮我审查这段代码"
  ↓
AI 看到 "审查" 关键词
  ↓
匹配到 code-review Skill 的 description
  ↓
自动加载并应用这个 Skill
  ↓
整个过程中用户不需要手动指定任何东西
```

这套机制设计得很优雅——AI 替用户判断该用什么工具，用户只管提需求。但坦白说，自动触发在实际中并不总是可靠的，它受几个因素影响：

- AI 工具的实现——有的客户端对 Skill 的支持比较完善（如 Claude Code），有的则偏弱，触发率因人而异
- 大模型本身的能力——高端模型对意图的识别更准确，低端模型则容易"听不懂"

大模型本身的能力

- 工作环境的复杂度——同时加载了多个 Skill 时，AI 可能匹配错或漏匹配

工作环境的复杂度

- description 的清晰度——这个下面会专门展开

description 的清晰度

所以，当自动触发不生效时，手动调用 Skill 是完全合理的做法。Slash 命令 /skill-name 就是为此设计的。

```
/code-review 帮我审查这段代码
/humanizer 帮我去掉AI痕迹

1
2
```

手动触发不是错误，而是自动触发的有效补充。

一句话总结：自动触发是理想，手动触发是后路。它们之间不是对错关系，而是互补关系。

触发决策过程

AI 在决定是否触发一个 Skill 时，大致经过以下判断链：

```
1. 分析用户意图
   └ 用户想做什么？

2. 检查 Skill 库
   └ 哪些 Skill 的 description 可能匹配？

3. 相关性评分
   └ 这个 Skill 对当前任务的帮助程度

4. 决定是否加载
   └ 如果相关性高，则加载 Skill 正文

5. 执行任务
   └ 按照 Skill 的指导完成任务
```

这个过程中，步骤 2 和 3 是最不稳定的环节。如果 description 写得模糊，或者模型本身对语义理解不够好，匹配就可能失败。

几种典型情况

```
情况1: 明确匹配

用户: "帮我生成 Git 提交消息"
                ↓
        关键词: "Git 提交消息"
                ↓
   匹配 Skill: git-commit-message
   description: "生成规范的 Git 提交消息..."
                ↓
            触发
```

```
情况2: 隐含匹配

用户: "我改了登录功能,帮我写个 commit"
                ↓
        关键词: "commit"（隐含在描述中）
                ↓
   匹配 Skill: git-commit-message
   description: "...当用户提到 commit..."
                ↓
            触发
```

```
情况3: 不匹配

用户: "今天天气怎么样?"
                ↓
     关键词: "天气"
                ↓
  没有匹配的 Skill
                ↓
         不触发任何 Skill
```

```
情况4: 自动触发失败，手动补位

用户: "帮我写个 commit 消息"（说了但 AI 没反应）
                ↓
        手动输入:
        /code-review 帮我生成 commit 消息
                ↓
            正常触发
```

情况 4 在实际使用中很常见。碰到自动触发不生效时，直接用 / 命令调用就好，不需要纠结。

提高触发准确率

自动触发靠的是 description，所以 description 写得越精准，触发越可靠。

不够好的 description：

```
"代码审查工具"

1
```

问题：太简单，AI 不知道何时用；缺少触发关键词；没有说明使用场景。

优秀的 description：

```
"专业的代码审查助手，检查安全漏洞、性能问题、代码规范、
 最佳实践。当用户要求审查(review)、检查(check)、
 评估(evaluate)代码质量时使用"
```

优点：说明了功能（检查什么）；列出了触发关键词（review, check, evaluate）；明确了使用场景（代码质量检查）；帮助 AI 准确匹配。

编写技巧

```
Description 公式:

[功能描述] + [触发关键词] + [使用场景]

示例:
"Python 开发最佳实践,
 包括代码规范、类型注解、文档字符串。
 当编写、修改、审查 Python 代码时使用,
 也可用于代码重构和质量提升"
```

但即使是写得最好的 description，也不能保证 100% 触发。触发率始终受到工具实现、模型能力、上下文环境等因素影响。所以既要认真写好 description，又要接受手动触发作为正常的后路——这两者并不矛盾。

■ 学点英语

中文	English	音标	说明
YAML元数据区	YAML Frontmatter	/ˈjæml frʌnt ˌmætər/	SKILL.md顶部的元数据区，用---包围，包含name和description两个必需字段
名称字段	Name	/neɪm/	Skill的唯一标识符，只能使用小写字母、数字和连字符，最多64字符
描述字段	Description	/dɪˈskrɪpʃn/	Skill的触发入口，AI靠它判断何时加载，需包含触发关键词
触发机制	Trigger Mechanism	/ˈtrɪgər ˈmekənɪzəm/	AI根据用户意图自动匹配Skill的过程，先匹配description再加载正文

下一节预告

前面讨论的 Skill 都停留在文本指引层面——AI 读规则，然后自己执行。但遇到需要实际操作的任务（如文件转换、API 调用）时，光靠文本不够。下一节我们来探讨 Skill 如何调用工具和脚本，让 AI 不仅能"想"，还能"做"。

图片

